OPEN DESIGN & TECHNOLOGY

Week 11

THE BLACK BOX

Dissecting a Technology

Automatic Sliding Door

A door that decides when to open and when not to, without anyone telling it explicitly. What is it reading, and what rules govern the choice?

Car Cruise Control

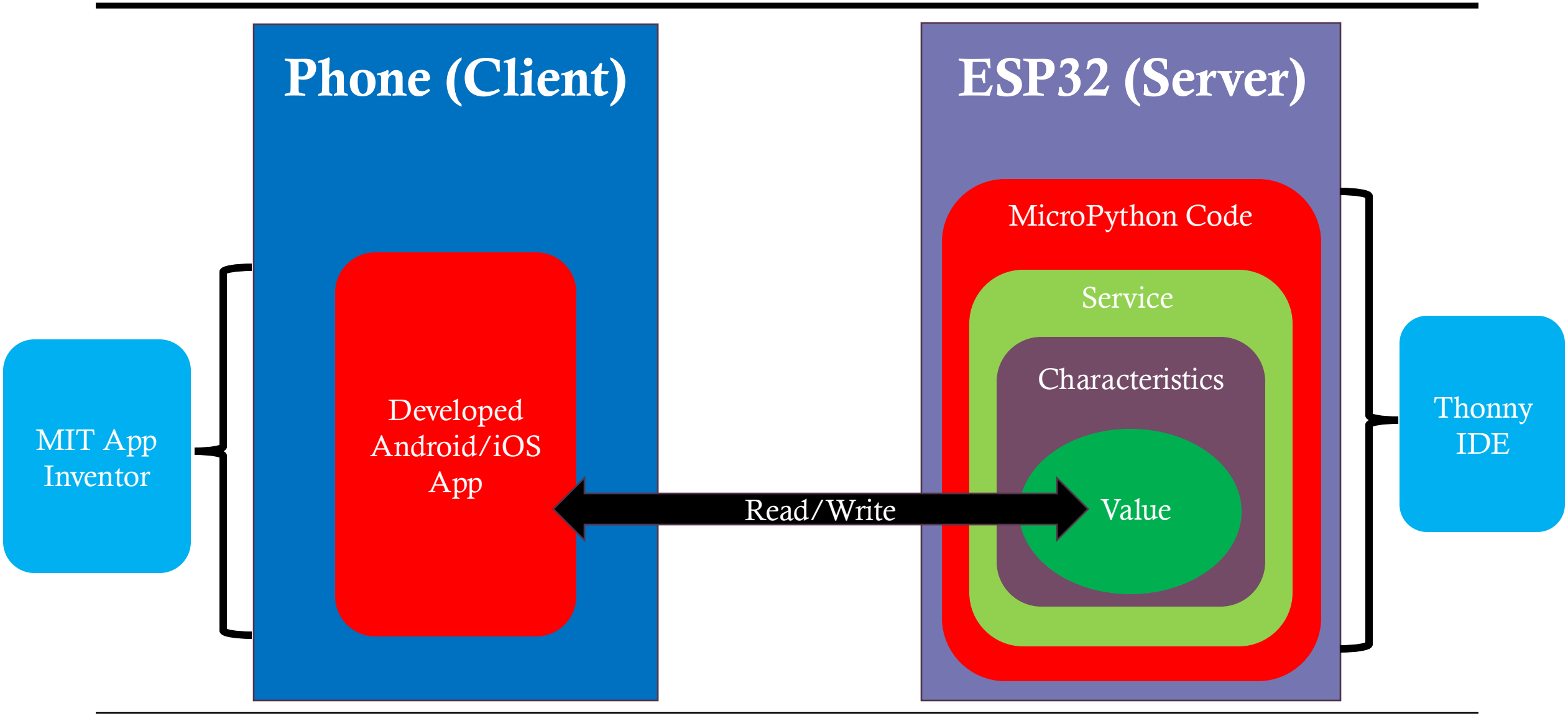
You set a speed and let go. The car maintains it across hills, headwinds, and curves, by constantly sensing a gap between what is and what should be.

Auto-Brightness on a Phone Screen

Your screen adjusts itself without asking. It reads something about your environment, but also about you. Where does sensing end ?

BLUETOOTH

Metaphor : Professor(s) + Student(s) + Bag(s) + File(s) + Paper(s)


Phone (Client)
ESP32 (Server)
MicroPython Code
Service
Characteristics
Value
Developed Android/iOS App
Read/Write
MIT App Inventor
Thonny IDE

BLUETOOTH PROJECT 1

Activity : Send & Write "Value" in "Characteristics" of the server

Controlling an OUTPUT using the App developed in MIT App inventor platform

STEP 1 : BUILDING THE **HARDWARE** OF THE **SERVER** (ESP32)

- Choose the Output (Servo, DC, Stepper or NeoPixel)
- Make the circuit connections : Interfacing the selected output with ESP32
- Connect a single LED (this will be used to indicate that the ESP32 is advertising / broadcasting)

STEP 2 : BUILDING THE **SOFTWARE** OF THE **SERVER** (ESP32)

- Decide on the functionality. Examples :
  - Control the Servo motor Position : left, right and center
  - Rotate the stepper motor clockwise & Counter-clockwise
  - Ligh up NeoPixel in different patterns
- **Change the name of your ESP32**
- Edit the code as per the designed functionality. Examples :
  - 1 = Servo Motor at 0 degree
  - 2 = Servo Motor at 90 degree
  - 3 = Servo Motor at 180 degree

STEP 3 : BUILDING THE **SOFTWARE** OF THE **CLIENT** (PHONE)

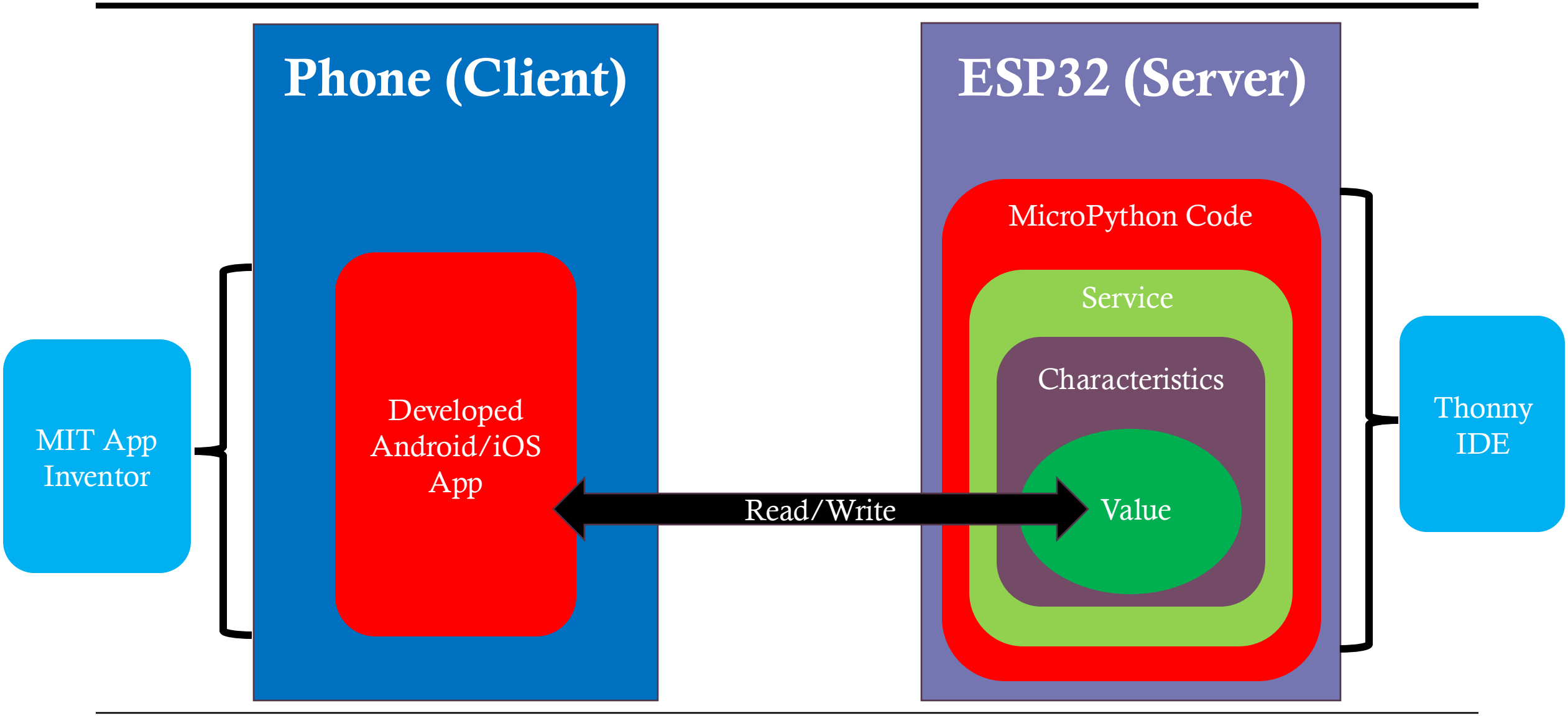
- **Design the UI of the app as per the designed functionality**
- Hardcode the button(s) to write the desired msg

Android / iOS App on Phone :

- Scan Button to start scanning (Scan for 3 seconds & stop)
- A Status label to display the status of the system (Ex : Scanning, Connecting, Connected, Disconnected)
- Display the list of devices advertising / broadcasting found during scanning.
- When a device is selected, an indicator label shows the name of the device
- Connect button to connect to the device
- Disconnect button to disconnect from the device
- Functionality Button(s) : To "write" "value" in the characteristics. (Ex : 1,2,3,a,c,e etc.)
  - Button 1 – Sends & Writes '1'
  - Button 2 – Sends & Writes '2'
  - Button 3 – Sends & Writes '3'

BLUETOOTH PROJECT 2

Activity : Only Reading "Values" from ESP32 server which generates data


Phone (Client)
ESP32 (Server)
MicroPython Code
Service
Characteristics
Value
Developed Android/iOS App
Read/Write
MIT App Inventor
Thonny IDE

STEP 1 : BUILDING THE **HARDWARE** OF THE **SERVER** (ESP32)

Since we do not have a Temperature Sensor, we write the code to generate Random Number which acts as Data generated by a Sensor

STEP 2 : BUILDING THE **SOFTWARE** OF THE **SERVER** (ESP32)

- Bluetooth Beacon Code from GitHub
- It Generates its own Sensor Data
- **Change the name of your ESP32**

STEP 3 : BUILDING THE **SOFTWARE** OF THE **CLIENT** (PHONE)

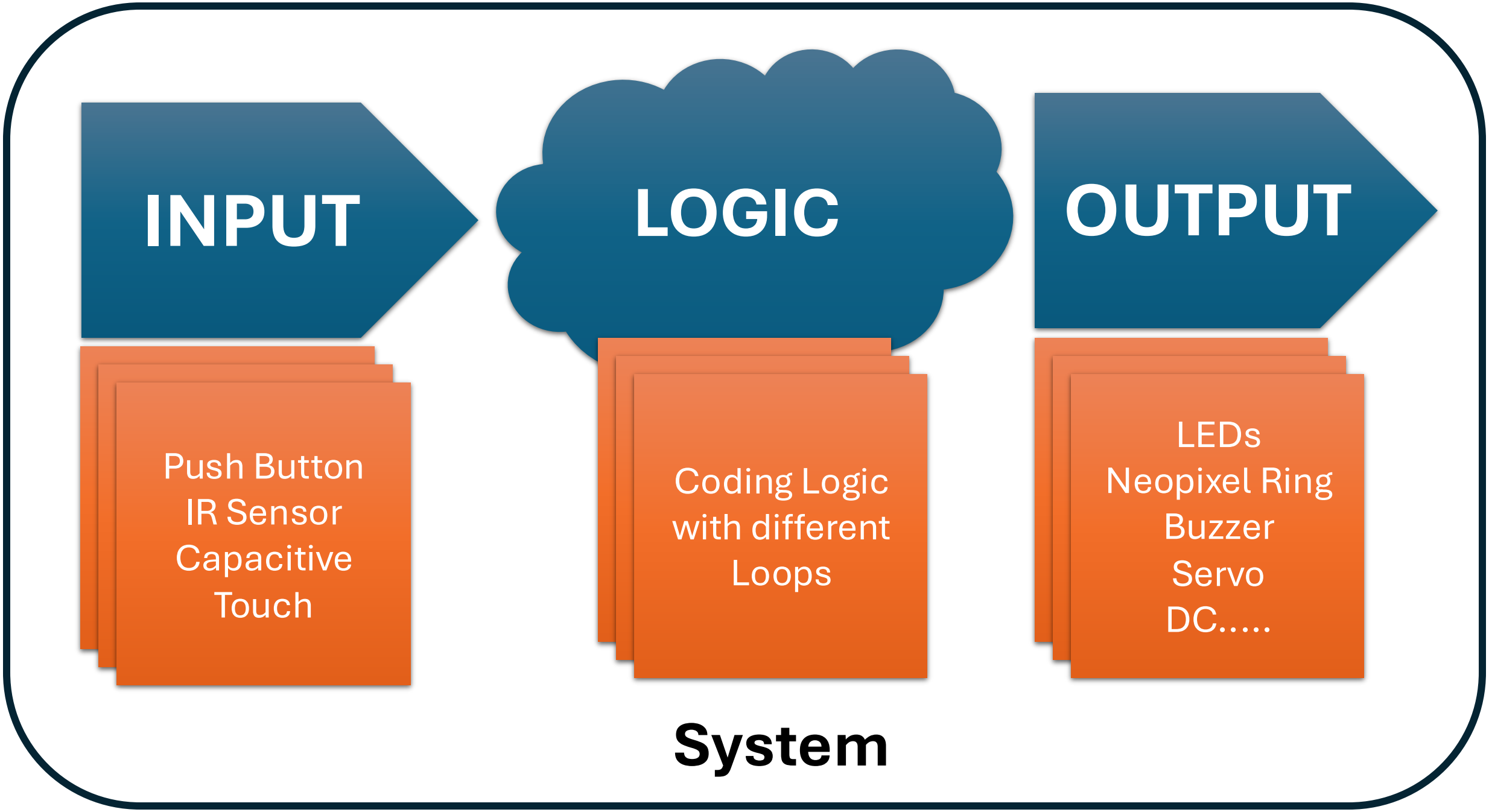
- **LightBlue for testing**

STEP 3 : BUILDING THE **SOFTWARE** OF THE **CLIENT** (PHONE)

- **MIT App Inventor**
- **Design the UI of the app as per the designed functionality**
- Add a Text Box to Display the data generated in the ESP32

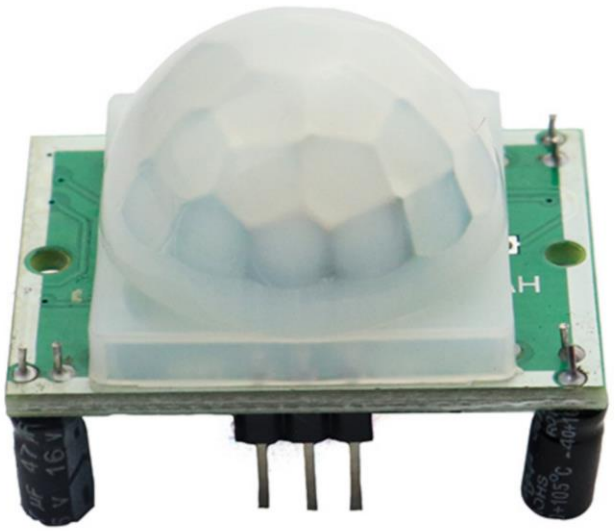
Android / iOS App on Phone :

- Scan Button to start scanning (Scan for 3 seconds & stop)
- A Status label to display the status of the system (Ex : Scanning, Connecting, Connected, Disconnected)
- Display the list of devices advertising / broadcasting found during scanning.
- When a device is selected, an indicator label shows the name of the device
- Connect button to connect to the device
- Disconnect button to disconnect from the device
- Add a Text Box to display the Sensor Values in ESP32 server's "characteristics" every 1 second


INPUT
LOGIC
OUTPUT
Push Button
IR Sensor
Capacitive
Touch
Coding Logic
with different
Loops
LEDs
Neopixel Ring
Buzzer
Servo
DC.....
System

IR OBSTACLE SENSOR??

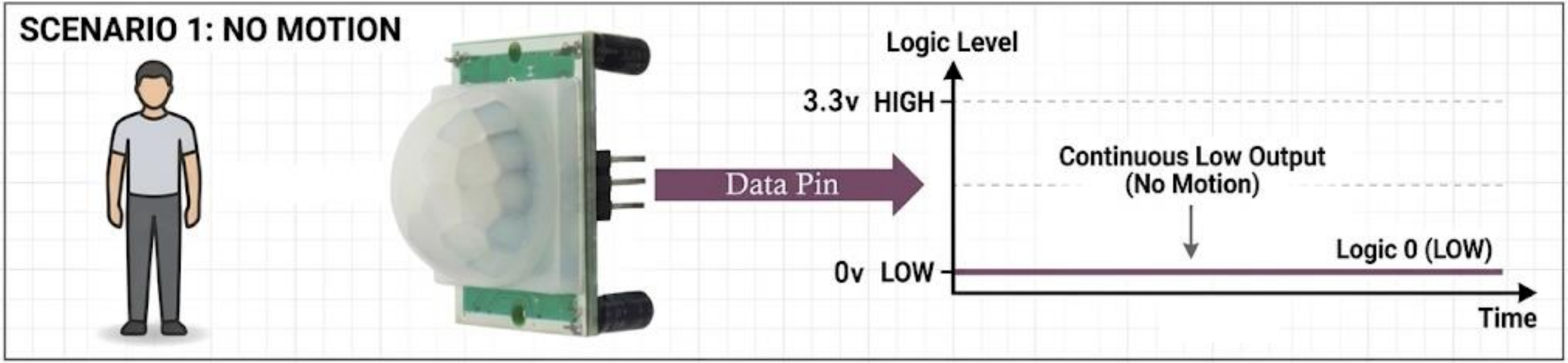
HC-SR501 PIR SENSOR



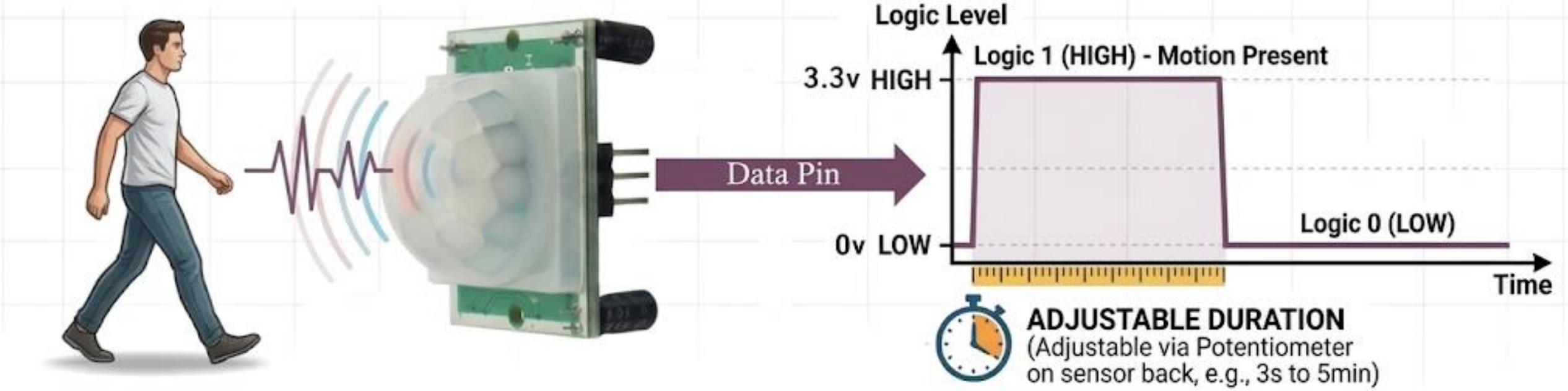
Motion Detection

HC-SR01 PIR MOTION SENSOR OPERATIONAL PRINCIPLE

SCENARIO 1: NO MOTION



SCENARIO 2: MOTION DETECTED



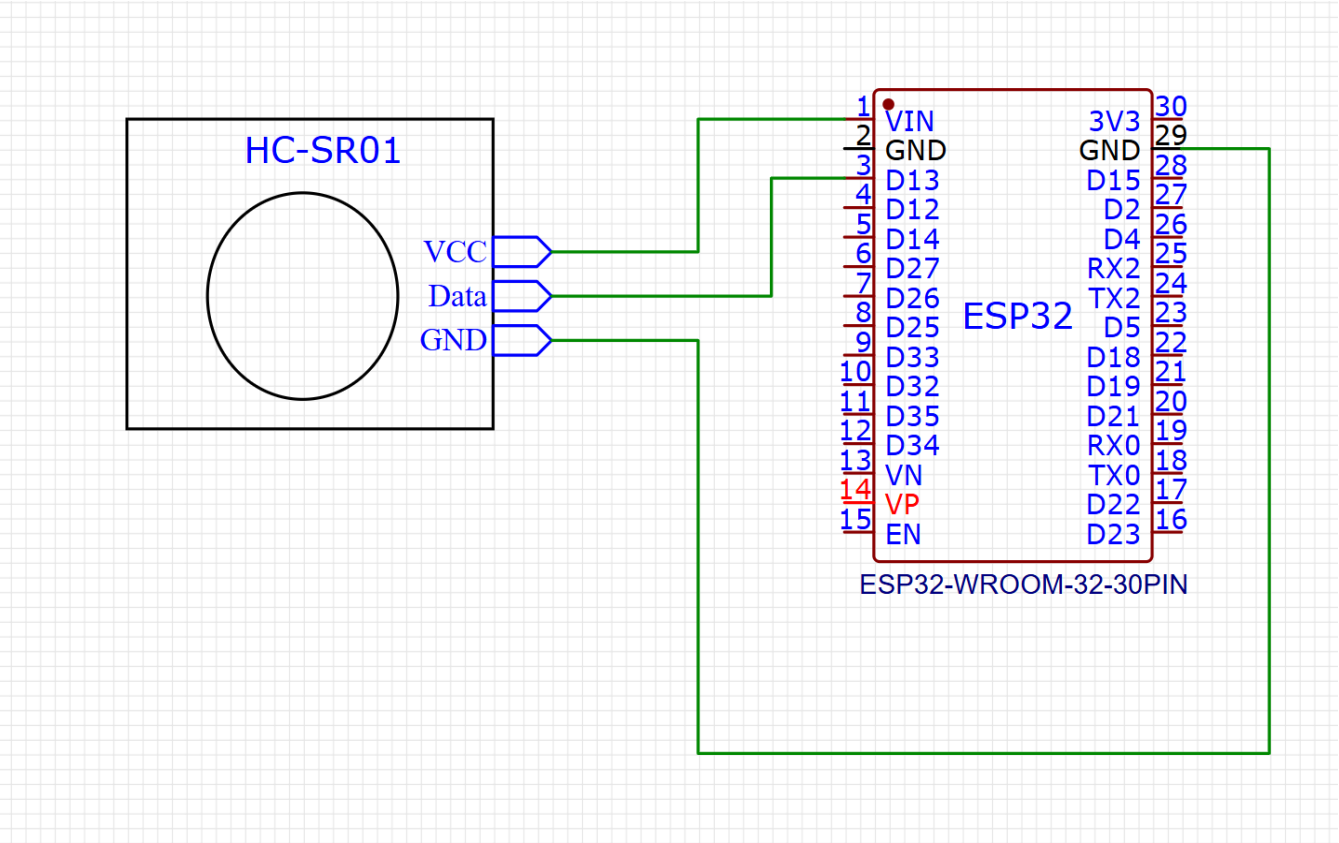
Counter-clockwise to decrease
Clockwise to increase

To Adjust **Output Hold Time**
To Adjust **Distance/Sensitivity**

Rotate both knobs counter-clockwise to set **Output Hold Time & Sensitivity to Minimum**

IMPORTANT TO NOTE

- **Warm-up :** On power-up, the sensor takes 30–60 seconds to stabilize its baseline IR reading. During this window it will fire false triggers repeatedly. Always build a startup delay in code.
- **The "Dead Time":** After the output goes LOW, there is often a 2-3 second "blind" period where the sensor will not trigger again.
- **Detection range** : 3 – 7 m (adjustable)
- **Detection angle :** ~110° cone
- **Output hold time** : 5 s – ~5 min (adjustable). Factory setting is of 5mins, reduce by rotating counter clock-wise



Import the required Libraries

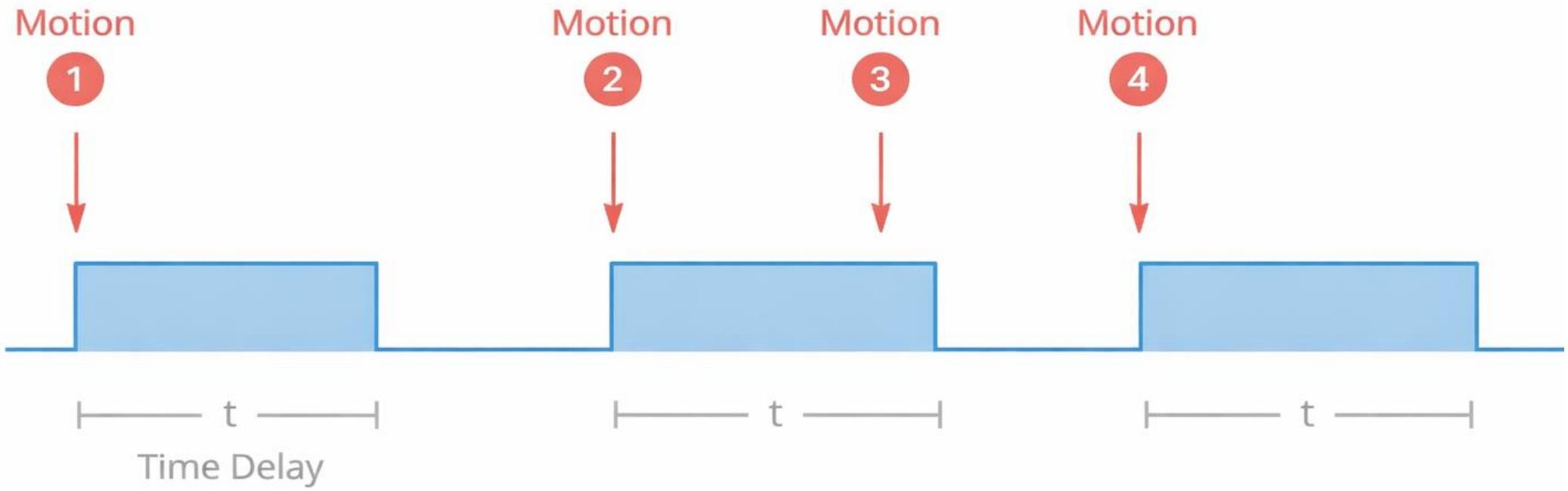
Name & Initialize the Sensor (Will it be an Input or Output??)

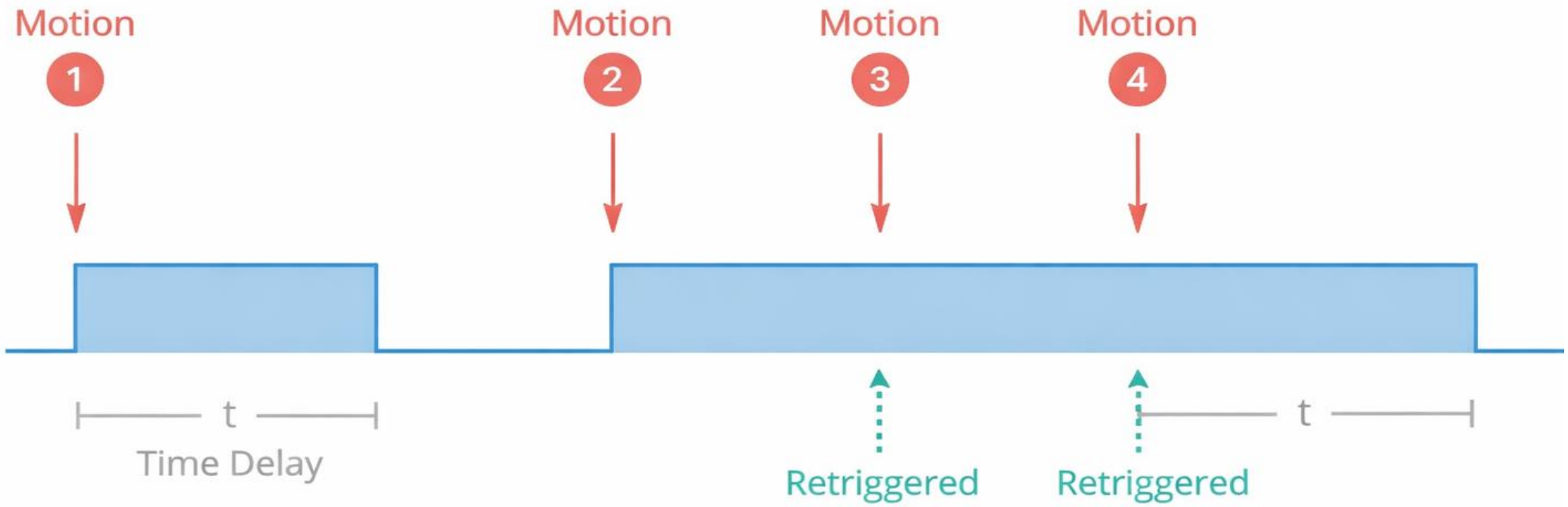
The sensor needs a warm-up time of 60 seconds

Read & print the sensor value continuously

If Motion detected, print "Motion Detected"

Delay of 0.5 second for stability


Motion
1
Motion
2
Motion
3
Motion
4
t
Time Delay
t
t


Motion
1
Motion
2
Motion
3
Motion
4
t
Time Delay
Retriggered
Retriggered
t

Pin.IN,Pin.PULL_UP

MC – 38 Wired Door Sensor

If :

$0 \rightarrow 0$

$5 \rightarrow 500$

What would be the value of :

$2.5 \rightarrow ?$

$1 \rightarrow ?$

If :

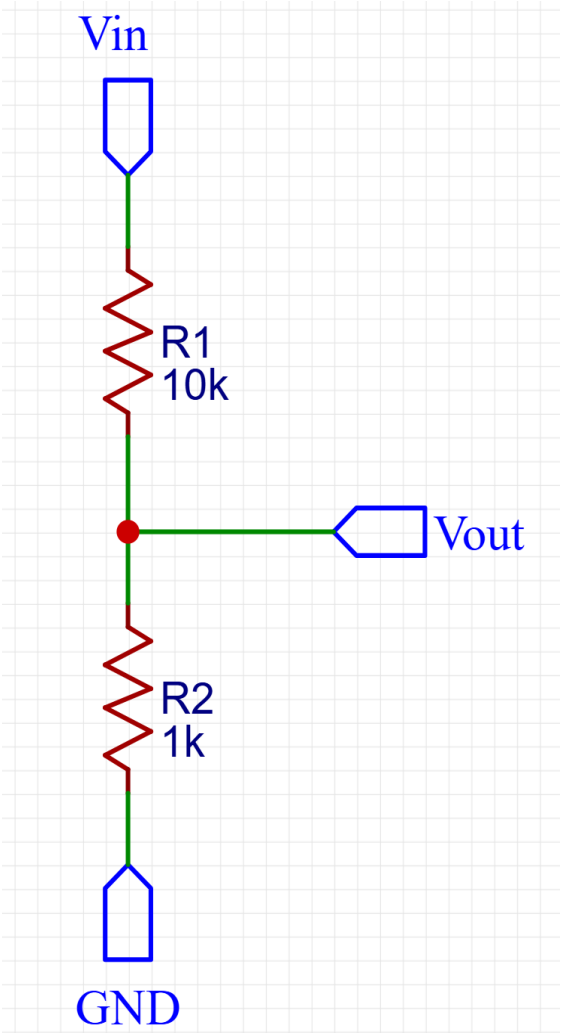
$0 \rightarrow 0$

$3.3 \rightarrow 4096$

What would be the value of :

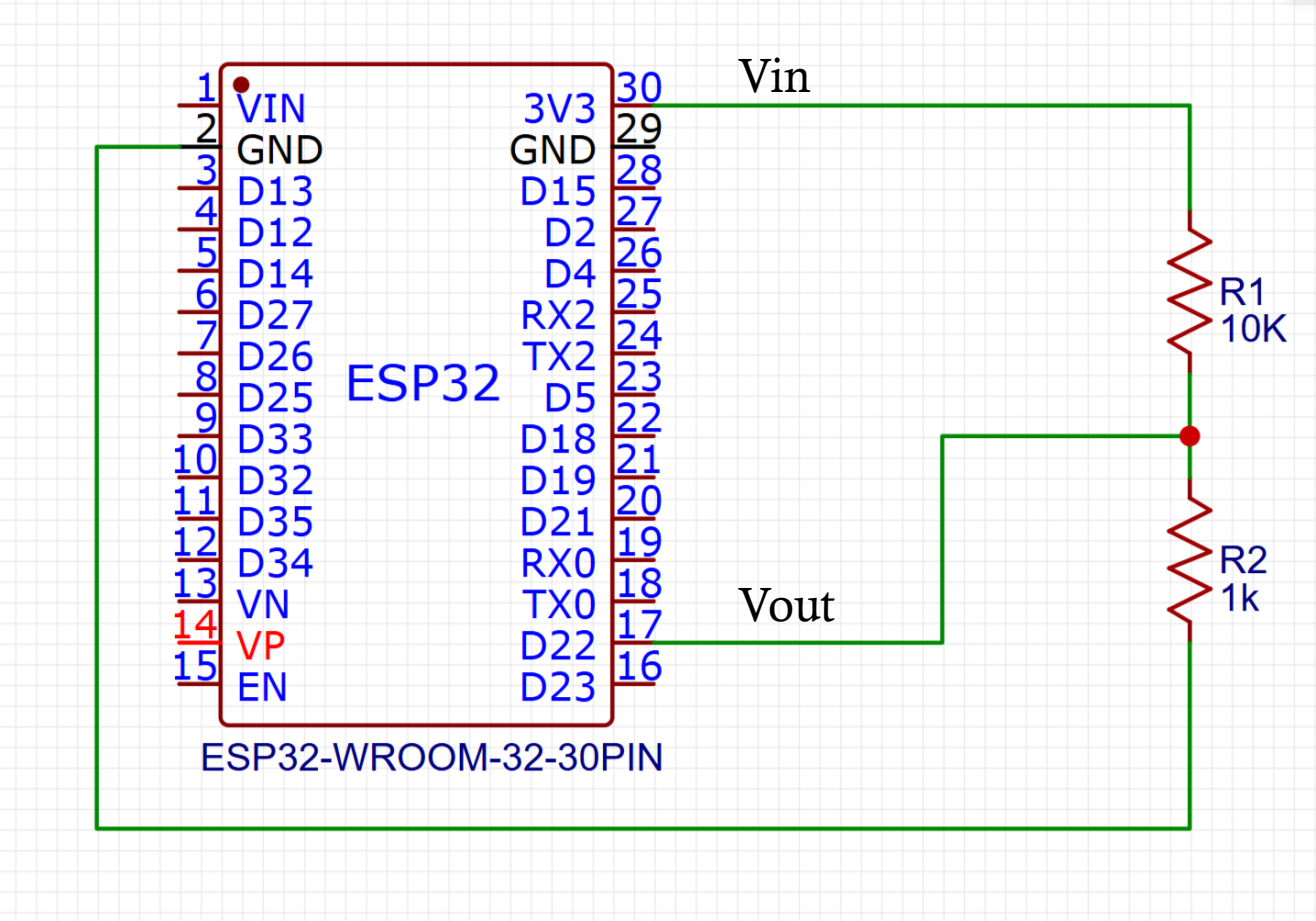
$2.5 \rightarrow ?$

$1 \rightarrow ?$



Vin	R1	R2	Vout
3.3v	10k Ω	1k Ω	0.3
3.3v	10k Ω	10k Ω	1.65
3.3v	10k Ω	60k Ω	2.82
3.3v	10k Ω	110k Ω	3.025

$$V_{out} = V_{in} \cdot \left(\frac{R_2}{R_1 + R_2} \right)$$



R2	Vout	ESP32
1k Ω	0.3	≈ 370
10k Ω	1.65	≈ 2047

Fixed value :

R1 = 10k Ω

Vin = 3.3v

MEASURING CHANGING RESISTANCE

BEND/FLEX SENSOR

Changing Resistance & ADC

Replace R2 with the Sensor

Precautions While Using a Flex Sensor

Avoid Excessive Bending
Do not bend the sensor beyond its specified limit (usually ~45°–90°). Over-bending can permanently damage it.

Do Not Fold or Crease
The sensor should be bent smoothly, not sharply folded. Creasing can break the conductive layer inside.

LEARNING OBJECTIVES : WEEK 11

- Dissecting a Technology (How do you perceive a Technology?)
- Bluetooth in detail
- Building a Bluetooth Project – Server, Client
- Motion detection using HC-SR01
- Understanding changing resistance & ADC
- Flex/Bend Sensor